Version 0.3.0

# Presentate

Simple dynamic presentation in **Typst**,
with flexible package integration framework.

## User Manual

GitHub: https://github.com/pacaunt/typst-presentate
Compiled on 11 October 2026

# 1 Contents


2 Introduction ..... 3
2.1 Acknowledgements ..... 3
3 Getting Started ..... 4
3.1 Creating slides ..... 4
3.2 Simple animation ..... 4
3.3 Control the show and hide: index specification ..... 5
3.4 Jumps and Markers ..... 6
3.5 Time traveling with relative indices ..... 7
3.6 Specific functions for `list` and `enum` ..... 9
4 Advanced Usage ..... 10
4.1 Groupping pauses: the `fragments` function ..... 10
4.2 Change the hiding method: the `hider` argument ..... 11
4.3 Change the appearance of the content: the `transform` function ..... 11
4.4 Precise control of the animation: `motion` and `tag` function ..... 12
4.5 Advanced `motion`'s control command specification ..... 14
5 Package Integration Framework ..... 15
5.1 Integration by parsing: `interface` and `bridge` functions ..... 15
5.2 Animate the inanimate: a custom way to hide ..... 16
6 Configuration Options ..... 18
7 Exposed Utilities ..... 18
7.1 Internal element representation ..... 18
7.2 Internal states ..... 18
8 Themes ..... 19
9 Function Reference ..... 22
9.1 Animation Functions ..... 22
9.2 Package Integration Interface ..... 33
9.3 Presentate Slide Function ..... 34
9.4 Special Function in `animation` module ..... 35

# 2 Introduction

`Presentate` is a Typst package for creating *dynamic* PDF presentation that is compatible with other packages. The word *dynamic* means the compiled PDF contains *animated* content. But PDF is a static document format, how can it contains animations? Presentate will look into the content, and create a set of pages that reveal or hide some content based on the current number of frames called *subslides*, so that when going through the pages, it seems like the content is showing or hiding like a simple animation.

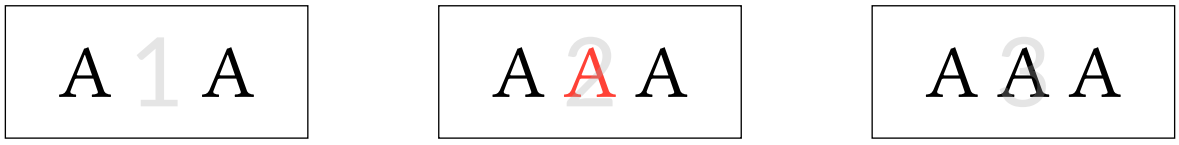
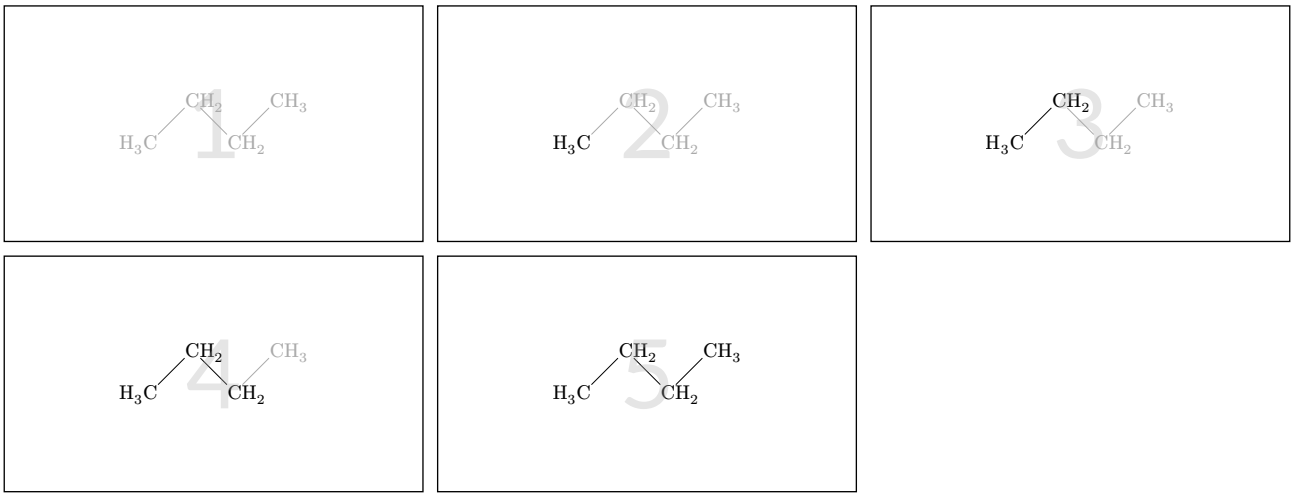



**Figure 1:** Example of fake animation in PDF format. Each page contains its own content and rules controlling them to hide, show, or decorated in some ways.

Dynamic PDF presentation packages have been in the Typst community for a long time, with the similar strategy for creating the animations, so why Presentate existed? The answer lies on the fact that presentation contains many kinds of visualization like plots, drawings, or chemistry molecules. These elements are usually created from other packages which have their own functions and data types. Therefore, to create animation with these visualization, the presentation framework must be able to handle *any* data types:

- be able to *hide* or *show* any elements, not only `content`,
- be able to generate the number of required subslides *automatically*.

Presentate is created for that purpose. It provides not only the slide animation engine, but also a framework and functions to interact with content from other packages. For example, here is some animation of drawing molecules integrated with the Alchemist[1] package.



Note that the gray text represents the subslide (frame) of that page.

## 2.1 Acknowledgements

The package was created by mixing my original motivation and insprations from many existing presentation packages. Thanks to:

- Polylux[2] for `subslide` implementation and pdfpc support,
- Touying[3] for idea of content-parsing, render frame, fake frozen states, and

[1] https://typst.app/universe/package/alchemist
[2] https://github.com/polylux-typ/polylux

- Minideck[4] for `only`, and `uncover` functions.
- Navigator[5] for the navigation system, progressive-outline, and the structured themes.

# 3 Getting Started

Import the package by

```
#import "@preview/presentate:0.3.0": *
```

Or download a zip file of Presentate's repository[6], install the `src` folder and import the package by

```
#import "src/export.typ": *
```

Then, setting your own styles, for example,

```
#set page(paper: "presentation-16-9") // you can change to 4-3
#set text(size: 25pt)
// your presentation begins here!
```

## 3.1 Creating slides

A slide can be created by using the `slide` function. For example,

```
#slide[
  Hello, this is presentate.
]
```

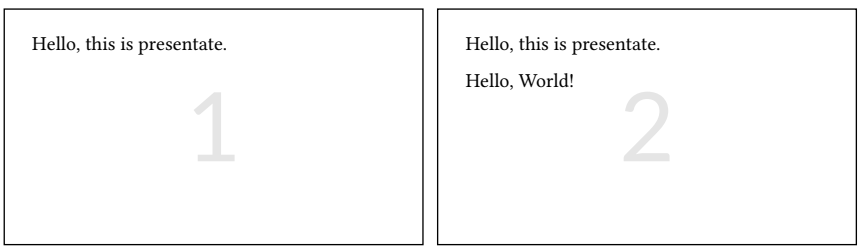
## 3.2 Simple animation

In the slide function, you can use the animation functions to control the behavior of the content on the slide. The simplest animation is to show some content after another content, this can be done by using `pause` marker:

```
#slide[
  Hello, this is presentate. #pause

  Hello, World!
]
```

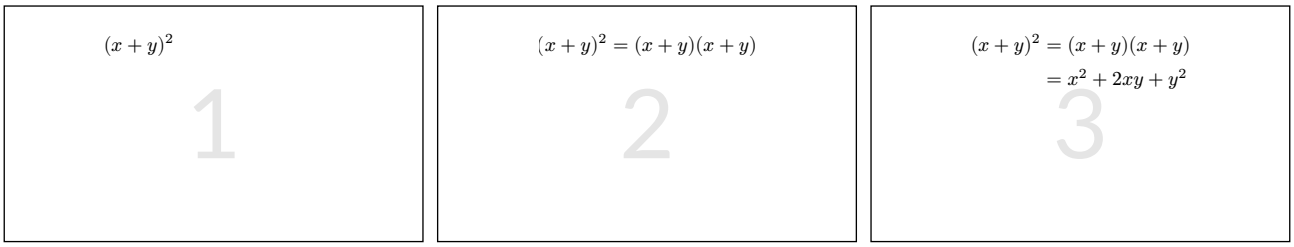
yields




`#pause` mark hides all the content coming after it, and incrementally reveal them frame-by-frame. It can be used in `math.equation` too.

[3] https://github.com/touying-typ/touying
[4] https://github.com/knuesel/typst-minideck
[5] https://github.com/eusebe/typst-navigator
[6] https://github.com/pacaunt/typst-presentate

```
#slide[
  $
    (x + y)^2 pause &= (x + y)(x + y) \
                pause &= x^2 + 2x y + y^2
  $
]
```

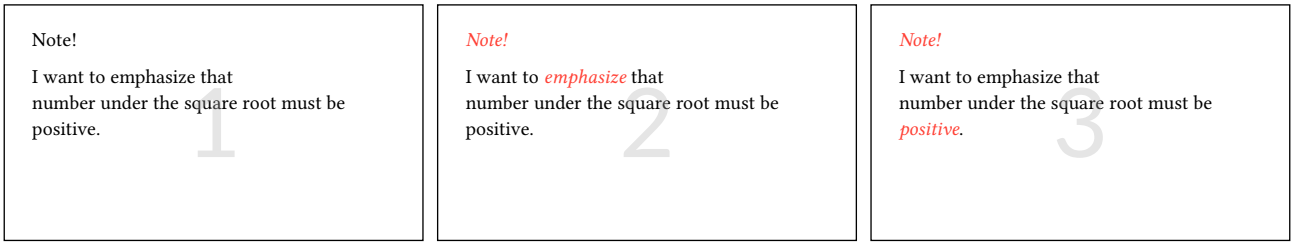



## 3.3 Control the show and hide: index specification

If, for example, you want to emphasis a text on a slide, you can use the `alert` function. This function, by default, will *emphasize* the content by wrapping it in the typst's `emph` function and become *alerted* on the specified subslide.

```
#show emph: set text(fill: red) // for apparent looking
#slide[
  #alert(from: 2)[Note!] // stays alerted from subslide 2

  I want to #alert(2)[emphasize] that \
  number under the square root must be #alert(3)[positive].
]
```




In the example, `alert` function accepted an integer and it made the text red only when the subslide number is the same as the specified number. This is called **index specification** where the index (pl. indices) is the subslide number or *frame* number. Indices can be specified in many ways:

- It can be an integer, to alert the content only at that subslide.
- It can be `from: int`, to start alerting since the `int`$^{\text{th}}$ subslide.
- It can be multiple integers like `alert(1, 4, 5, body)` to only alert on subslide 1, 4, and 5.
- It can be *both* integers and `from: int`, to show the modified content on the specified subslide and after the `int` subslide specified by the `from` argument.

*added in 0.3.0: following Touying's functionality*

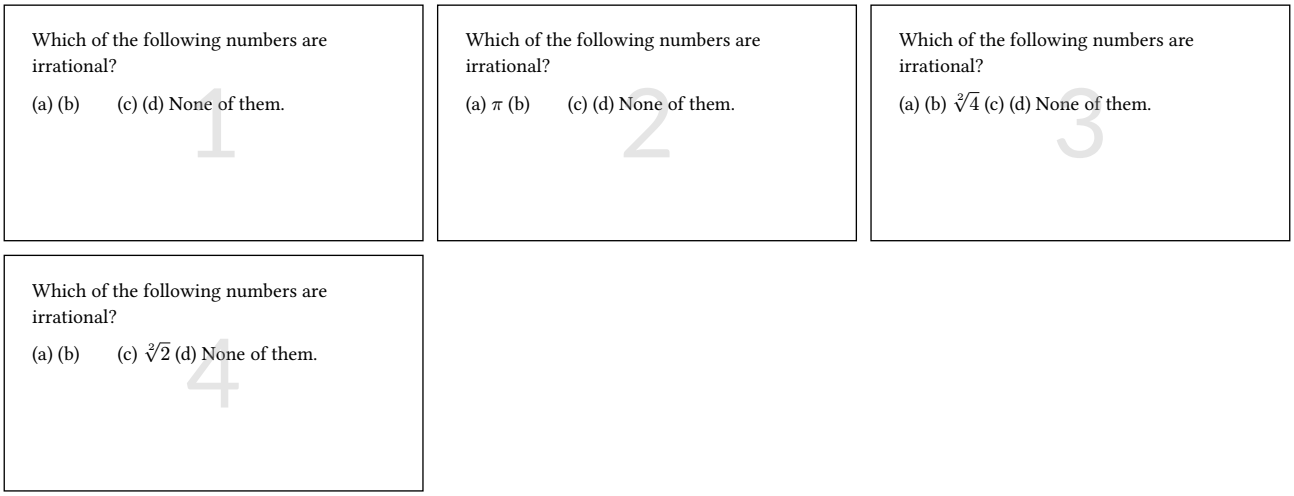
- It can be a string/label, specifying the index at which the label is present in `marker`. See Section 3.4.

Apart from `alert`, if you have ever used `beamer` in LaTeX before, you probably know the `uncover` and `only` functions. The `uncover` only *uncover* the content on the spcified subslides, and hide the content otherwise, with *space preserved*, while `only` function also works with the same logic, but its hiding method is to completely remove the content out, so no space is preserved.

```
#slide[
```

```
  Which of the following numbers are irrational?

  (a) #only(2, $pi$)
  (b) #uncover(3, $sqrt(4)$)  // The space here is peserved.
  (c) #only(4, $sqrt(2)$)
  (d) None of them.
]
```




## 3.4 Jumps and Markers

`jump(indices)` was introduced in 0.3.0, importing the functionality by Touying. This function returns a pause-like marker that set the current number of pauses to the specified `indices`.

```
A            // shown on subslide 1
#pause B     // shown on subslide 2
#jump(1) C   // shown on subslide 1
```

Moreover, you can mark a progress according to the current number of pauses by using `marker(<name>)`. This name can be used like the other indices.

```
#slide[
  = Combination
  Derive an equation of displacement vs time for a free-falling object from height $h$
at initial velocity of $u$.
  #pause #marker(<solution>)

  *Solution.* #pause
  $
    v = u - g t quad "and" quad s = ((u + v)/2) t \
    #pause
    s = ((u + (u - g t))/2) t quad pause ⇒ quad s = u t - 1/2 g t^2
    #marker(<end>)
  $
  #jump(<solution>)
  #uncover(from: (to:<solution>, rel: 1), to: <end>)[_Comments:_]
  #only(auto, update: true)[From definitions.]
  #only(auto, update: true)[Distribute the terms.]
  #only(<end>)[That's it.]
]

```

**Combination**
Derive an equation of displacement vs time for a free-falling object from height $h$ at initial velocity of $u$.

1

**Combination**
Derive an equation of displacement vs time for a free-falling object from height $h$ at initial velocity of $u$.

**Solution.**

2

**Combination**
Derive an equation of displacement vs time for a free-falling object from height $h$ at initial velocity of $u$.

**Solution.**

$$v = u - gt \quad \text{and} \quad s = \left(\frac{u+v}{2}\right)t$$

3

*Comments:* From definitions.

**Combination**
Derive an equation of displacement vs time for a free-falling object from height $h$ at initial velocity of $u$.

**Solution.**

$$v = u - gt \quad \text{and} \quad s = \left(\frac{u+v}{2}\right)t$$

$$s = \left(\frac{u+(u-gt)}{2}\right)t$$

4

*Comments:* Distribute the terms.

**Combination**
Derive an equation of displacement vs time for a free-falling object from height $h$ at initial velocity of $u$.

**Solution.**

$$v = u - gt \quad \text{and} \quad s = \left(\frac{u+v}{2}\right)t$$

$$s = \left(\frac{u+(u-gt)}{2}\right)t \quad \Rightarrow \quad s = ut - \frac{1}{2}gt^2$$

5

*Comments:* That's it.

## 3.5 Time traveling with relative indices

Most of the time when creating presentation, the step of revealing the content is either

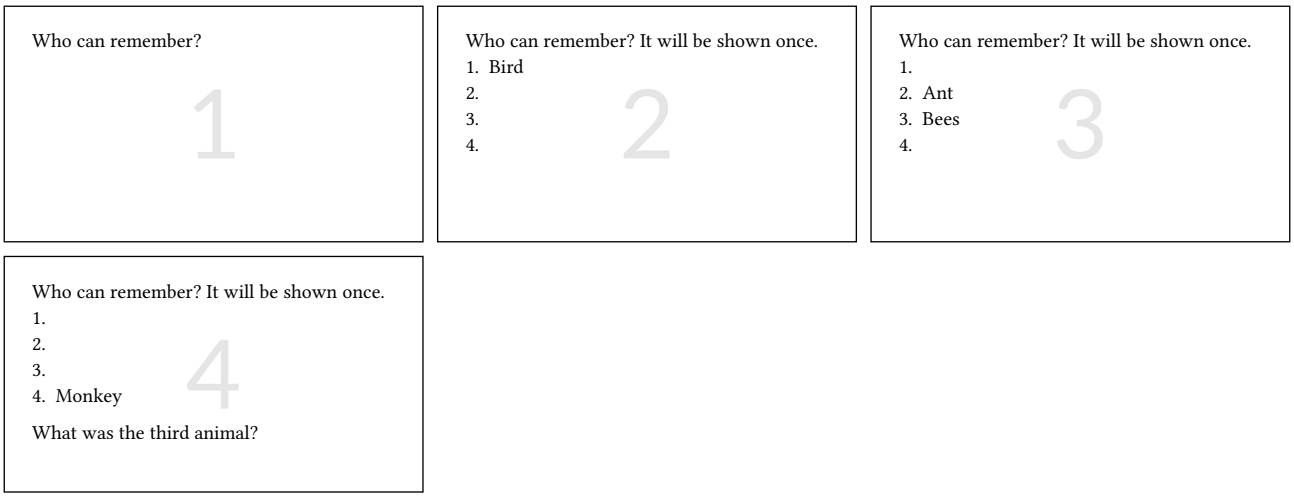
- showing at the same subslide as previous animation, or
- showing at the next subslide of the recent animation.

Presentate have a special type of index called **relative indices**, which are indices relative to the current number of pauses appear on the slide. There are 3 forms of relative indices:

1. `none` means the animation will happen at the same subslide as the current pause.
2. `auto` means the animation will happen at the *next* subslide after the current pause.
3. `(rel: int)` means the animation will happen at the `int` subslide away from the current pause.

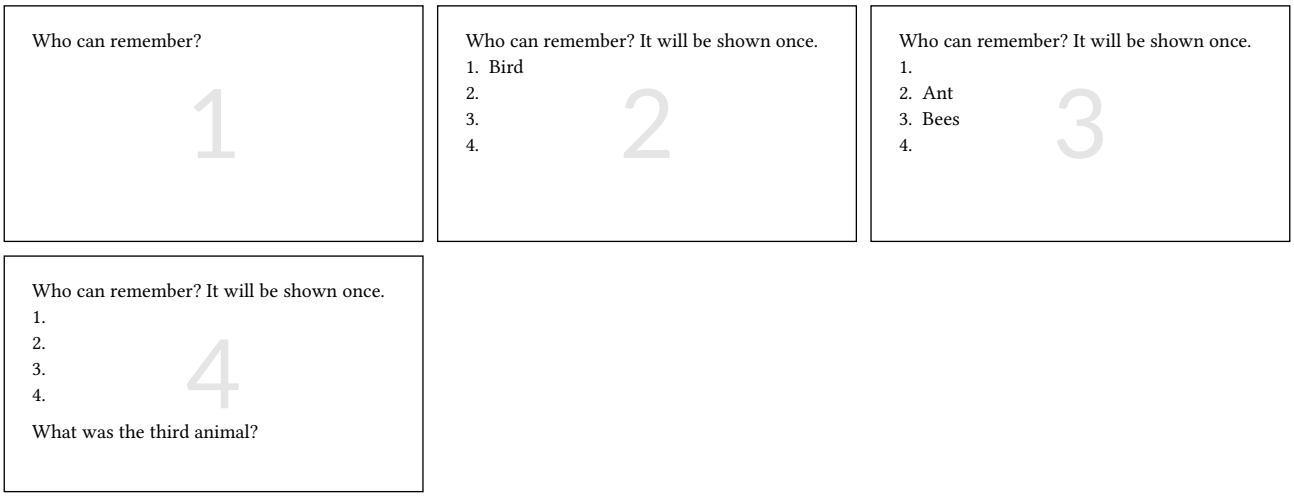
For example,

```
#slide[
  Who can remember?
  #pause It will be shown once. // first pause
  + #uncover(none)[Bird] // with the pause
  + #uncover(auto)[Ant] // after the pause
  + #uncover((rel: 1))[Bees] // also after the pause
  + #uncover((rel: 2))[Monkey] // two after the pause
  #uncover(from: (rel: 2))[What was the third animal?]
]
```




As you can see, the last text that was shown on subslide 4 was a little bit inconvenient to write `(rel: 2)` to make it shown when every animation was finished. It would be nice if we can use `pause` at the end and make it 'see' the previous `uncovers`. You can make the next pause reveal a content after (almost) any animation functions by specifying `update: true` into its argument.

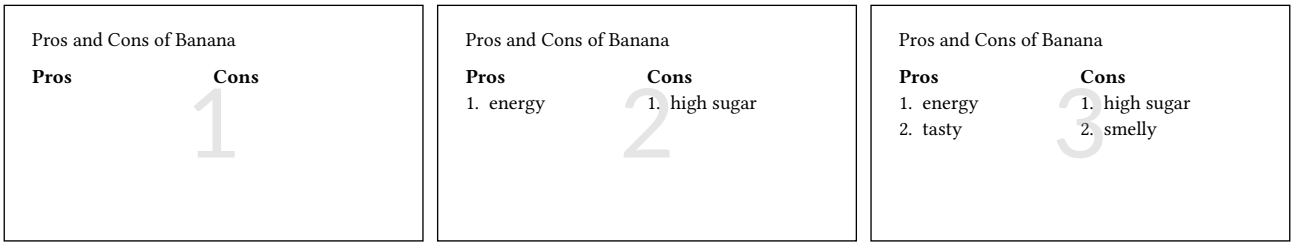
```
#slide[
  Who can remember?
  #pause It will be shown once. // first pause
  + #uncover(none)[Bird]
  + #uncover(auto)[Ant]
  + #uncover((rel: 1), update: true)[Bees] // update the last animation
  + #uncover((rel: 2))[Monkey]
  #pause What was the third animal? // so this shows later
]
```




With this interaction between the current number of pauses and the `update` argument, you can set the pauses to *any* number to modify the animation. For example, making the content shown in sync side by side:

```
#slide[
  Pros and Cons of Banana
  #grid(columns: (1fr,) * 2)[
    *Pros* #pause
    + energy
    #pause
```

```
  + tasty
][
  #meanwhile
  *Cons* #pause
  + high sugar
  #pause
  + smelly
]
]
```

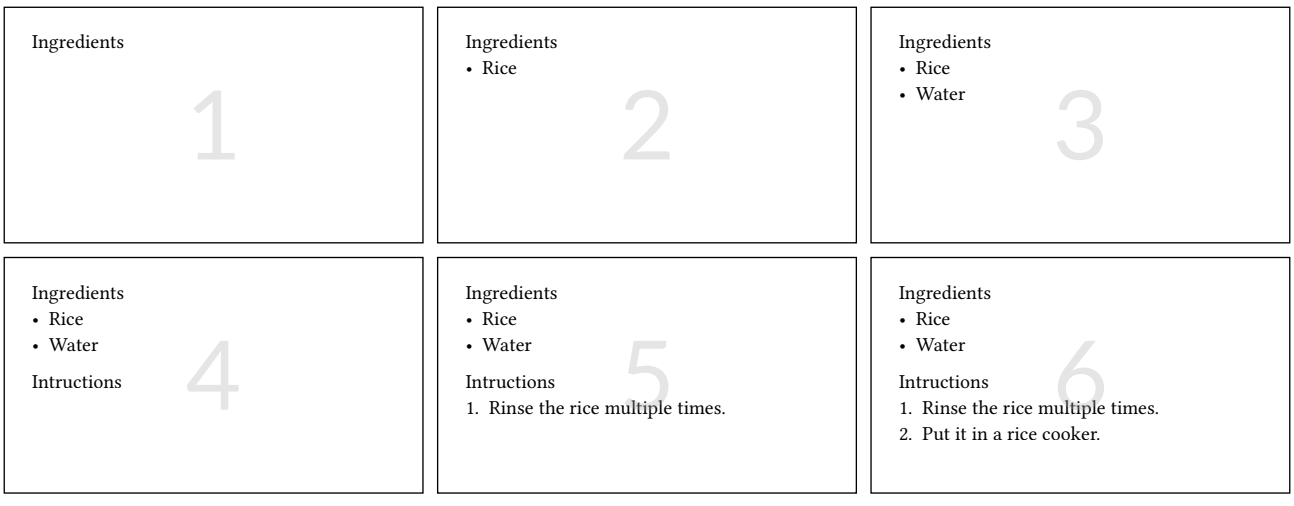



## 3.6 Specific functions for `list` and `enum`

*In presentate 0.3.0, `list` and `enum`'s markers are already hidden by `pause` or `jump`, so the following function is not needed.*

In the last example, you can see that the number in `enum` did not hide properly. Since Typst treats the marker of `list` and number label of `enum` in different context from its body, Presentate implements some (hacky) way to properly hide them without wiggling. To show the list/enum item by item, Presentate provides a function called `step-item`:

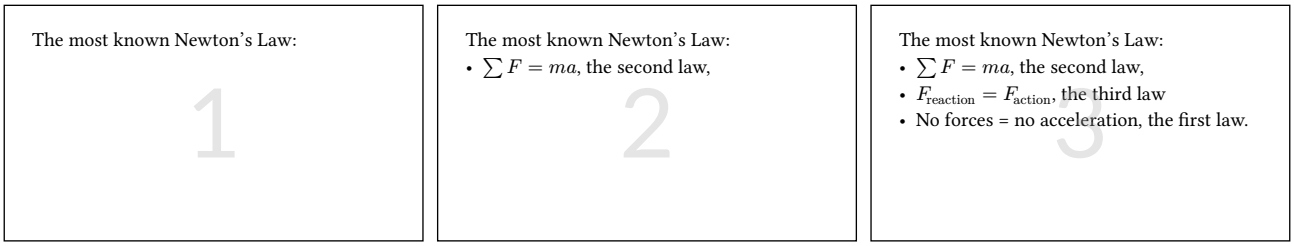
```
#slide[
  Ingredients
  #step-item[
    - Rice
    - Water
  ]
  #pause
  Intructions
  #step-item[
    + Rinse the rice multiple times.
    + Put it in a rice cooker.
  ]
]
```



Each of the item's body and its marker are wrapped in `pause` function, and synchronize the animation by using the `update-pause` hack. Therefore, you can nested them indefinitely, but it always update the pauses.

If you want to reveal the items chunk by chunk, Presentate also have the `reveal-item` function. This function can show item in group by group without breaking them. However, since the implementation of `reveal-item` requires additional Typst's state, it can decrease the speed of Typst compilation. So use this function sparingly.

```
#slide[
  The most known Newton's Law:
  #reveal-item[
    - $sum F = m a$, the second law,
  ][
    - $F_"reaction" = F_"action"$, the third law
    - No forces = no acceleration, the first law.
  ]
]
```




Note that both `step-item` and `reveal-item` function receive `start: auto` argument, which indicates when will the first item (or item group) be revealed.

# 4 Advanced Usage

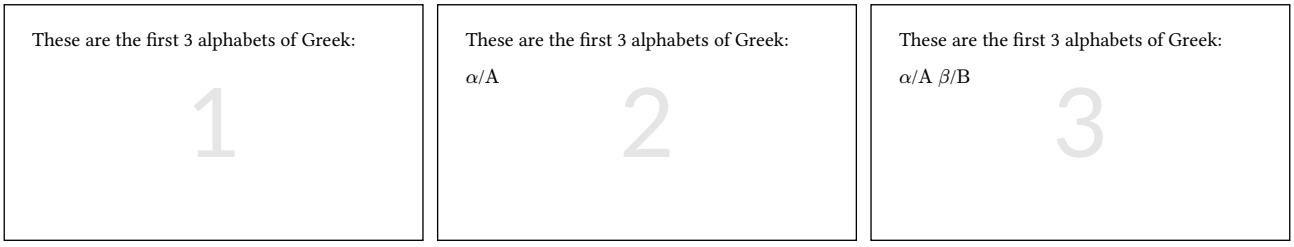
## 4.1 Groupping pauses: the `fragments` function

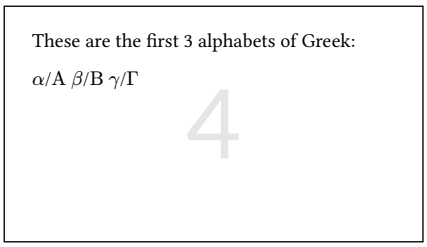
This function works like a shorthand for multiple pause/uncover calls. Moreover, you can choose whether to make it update the current pause by `update-pause` argument.

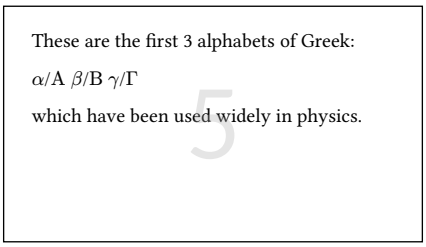
```
#slide[
  These are the first 3 alphabets of Greek:

  #fragments[$alpha$/$Alpha$ ][$beta$/$Beta$ ][$gamma$/$Gamma$]
  #pause

  which have been used widely in physics. // the fragments update the pause.
]
```






You can indicate when to start the animation by specifying `start: {index}`, which is `auto` by default.

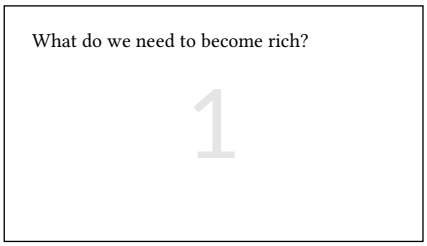
## 4.2 Change the hiding method: the `hider` argument

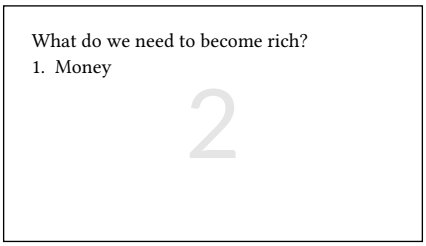
Every animation function in Presentation has a named argument `hider`, which is a function that used internally to *hide* the content. For example,

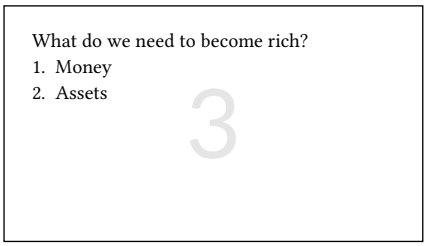
- `only` uses `it ⇒ none` as hider, which make the content disappear when it hides,
- `uncover`, `pause`, `step-item`, `fragments` uses native Typst's `hide` function, which preserves space.

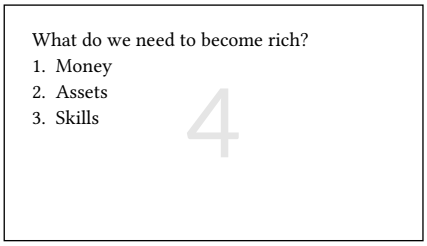
If you want to change the hiding method of the text from completely hiding it to making it gray, you can use `text.with(fill: gray.transparentize(50%))` as the hider, like the following example,

```
#slide[
  What do we need to become rich?
  #step-item(hider: text.with(gray.transparentize(50%)))[
    + Money
    + Assets
    + Skills
  ]
]
```



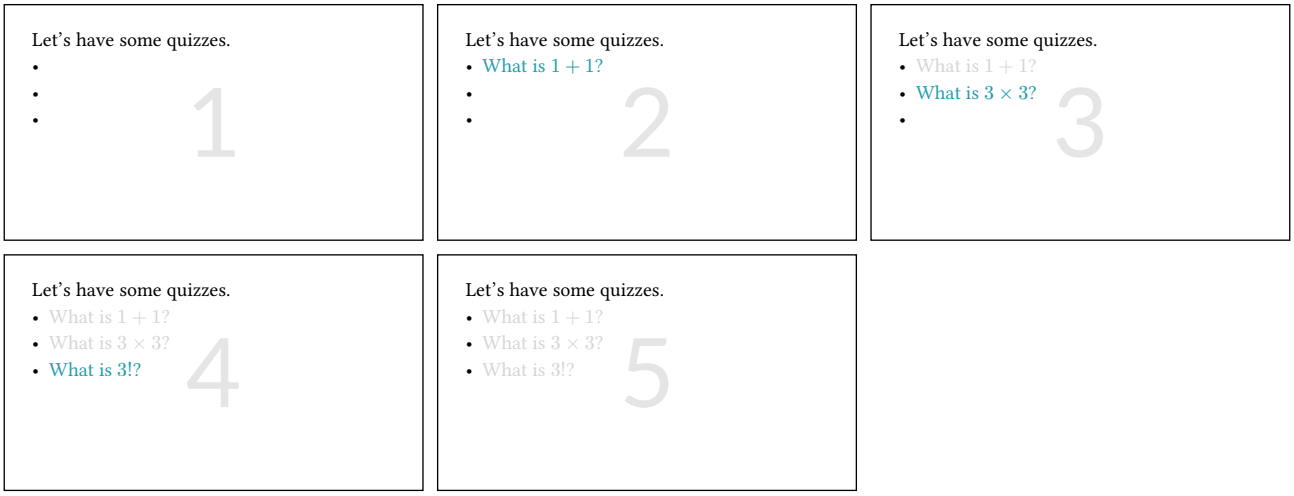




## 4.3 Change the appearance of the content: the `transform` function

`transform` requires a content as a base, it will be hidden by the `hider` until the specified index is reached (default is `auto`). Then, on each consecutive subslide, the content will transform by each function one by one. The following example shows how to emphasize content in a list by using `transform`.

```
#slide[
  #let yes = text.with(fill: eastern)
  #let no = text.with(fill: gray.transparentize(50%))
  Let's have some quizzes.
  - #transform([What is $1 + 1$?], yes, no)
  - #transform([What is $3 times 3$?], yes, no, start: none)
```

```
7   - #transform([What is $3!$?], yes, no, start: none)
8 ]
```




`start` argument tells when to start the animaton. Note that in this example, the grayed text was from `no` function, not the hider. The hider was still `hide` function, as you can see in the subslide 1.

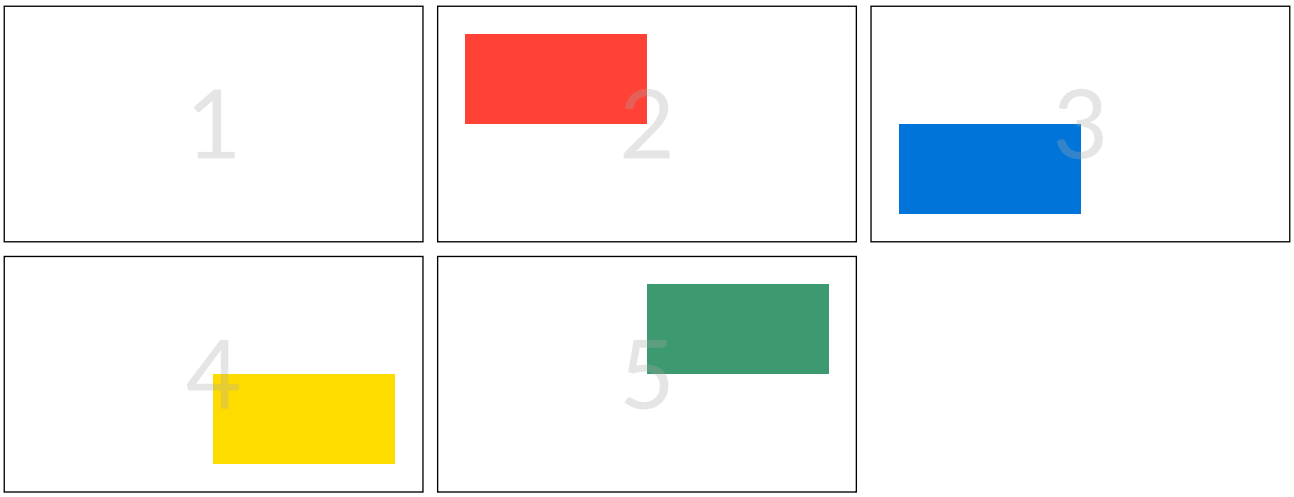
## 4.4 Precise control of the animation: `motion` and `tag` function

`motion` function provides a *workspace* for rendering content groups that are named by the `tag` function. The anatomy of this function is

```
1 #motion(s ⇒ [
2   // some content and `tag(s, name, body)`
3 ], controls: (
4   // rules to show the `tag`
5 ))
```

Motion function accepts only one positional argument which is a **function** that receives the presentate's state `s` and return a content block. `motion` function have a `controls` argument to specify which content group should be shown or hide precisely subslide by subslide. The motion reads the `controls` rules, process them and send the showing information through the state `s`. This state `s` is read by the `tag` function to show or hide the content accordingly.

```
 1 #slide[
 2   #motion(s ⇒ {
 3     set rect(width: 100%, height: 100%)
 4     grid(
 5       rows: 1fr, columns: (1fr,1fr),
 6       tag(s, "red", rect(fill: red)),
 7       tag(s, "green", rect(fill: olive)),
 8       tag(s, "blue", rect(fill: blue)),
 9       tag(s, "yellow", rect(fill: yellow))
10     )
11   },
12   controls: ((),"red", "blue", "yellow", "green"))
13 ]
```




This framework allows user to control the animation step without worrying about the location of each elements in the source code. For the specification of **controls** argument, here is the syntax:

- The `controls` only accept **an array of rules** that will be applied on each subslides, similar to `transform` and the functions.
- Each rule can be a single command or an array of commands that will be use at the same subslide.

For each command,

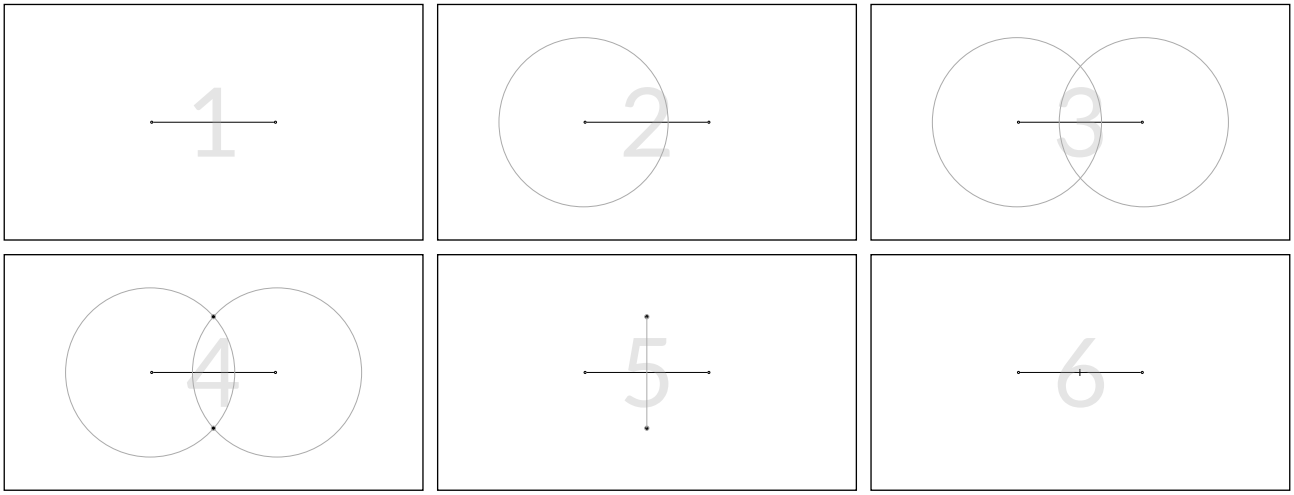
- You can provide a name of the tag to show the content **once**,
- or provide with a suffix `.start` to start showing the content from the current step,
- or with a suffix `.stop` to stop showing the content.

The following example is more complex and use all of the control rule specification.

```
#import "@preview/cetz:0.5.2": canvas, draw

#slide[
  #set align(center + horizon)
  #motion(
    s ⇒ [
      #canvas({
        import draw: *
        scale(1.5)
        stroke((thickness: 2pt))
        line(name: "L", (0, 0), (6, 0), mark: (symbol: "o"))
        stroke((paint: gray))
        intersections("I", {
          tag(s, "circ1", circle("L.start", radius: 4))
          tag(s, "circ2", circle("L.end", radius: 4))
        })
        tag(s, "pt", {
          circle(radius: .1, "I.1", fill: black)
          circle(radius: .1, "I.0", fill: black)
        })
        tag(s, "cut", line("I.0", "I.1"))
        tag(s, "mark", {
          stroke(black)
          mark("L.mid", "L.end", symbol: "|", width: .5)
        })
      })
    ],
    hider: draw.hide.with(bounds: true),
    controls: (
```

```
      (), // don't show any group
      "circ1.start", // show the `circ1`
      "circ2.start", // show the `circ2`
      "pt.start", // show the `pt`
      ("circ1.stop", "circ2.stop", "cut"),
      // stop showing `circ1` and `circ2`, and show the `cut` once
      ("mark.start", "pt.stop"), // show the `mark` and stop showing the `pt`
    ),
  )
]
```




This was created with integration of CeTZ[7] package, showing the capability to integrate with other packages. This integration works because of the `hider` argument in the motion function is the CeTZ's hide function, which is compatible for its own elements. Note that for more complex illustrations, you can provide custom `hider` to the `tag` function to use a specific hider for that group.

## 4.5 Advanced `motion`'s control command specification

For the version after 0.2.5, you can use a special syntax that let you modify the content in the tag group by applying some functions to it. The syntax of the command is

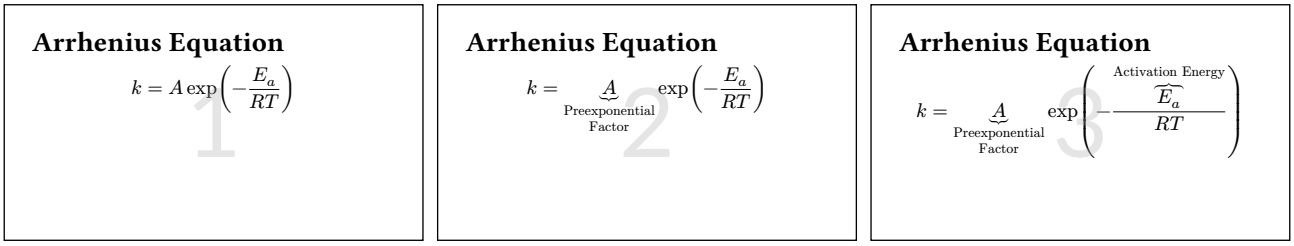
```
("{name}", {func}) // for apply the function once,
("{name}.start", {func}) // for starting showing the content and apply the function,
("{name}.apply", {func}) // for applying the function if it is showing,
"{name}.revert" // to revert back to original (identity) function
```

Here is an example of using this functionality: to annotate a mathematical expression:

```
#slide[
  = Arrhenius Equation
  #motion(
    s => [
      #let tag = tag.with(s) // a shorthand
      $ k = tag("A", A) exp(-tag("Ea", E_a)/(R T)) $
    ],
    controls: (
      ("A.start", "Ea.start", "exp.start"),
      ("A", it => $underbrace(it, "Preexponential\nFactor")$),
      ("Ea", it => $overbrace(it, "Activation Energy")$),
```

[7] https://typst.app/universe/package/cetz

```
    ),
  )
]
```




# 5 Package Integration Framework

Presentate's animation relies on parsing data in two formats: *content* and *array*, which are only two data types that can be joined intrinsically. Every parsable data, in Presentate's view, must be 1) reachable, meaning that **animation cannot be used in show-rules nor in `context` calls**, 2) recognizable, such as integer, content, string, etc. By design, Presentate transforms all contents into internal representation called `element`, the animation calls are just these elements wrapped by `metadata` or `array` so that they can join with the surrounding contents. From the limitation of 'reachable', Presentate overcomes this by introducing element constructors that can natively supports parsing of `array` and custom `content` datatypes: `interface` and `bridge`.

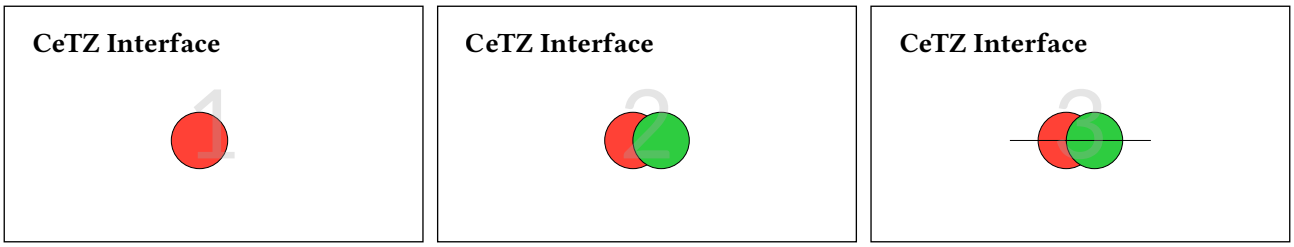
## 5.1 Integration by parsing: `interface` and `bridge` functions

To make Presentate reach the animation functions inside non-content environment, you must use `interface`, which splits the data and sends them to Presentate to process before rendering them back to its original environment.

```
#import "@preview/cetz:0.5.2": draw, canvas
// The wrapper
#let canvas = interface(
  canvas,                              // The environment function
  inner: "array",                      // datatype of the content inside
  hider: draw.hide.with(bounds: true) // hider used for `pause`
)

#slide[
  = CeTZ Interface
  // just for visuals
  #set align(center + horizon)
  #show: scale.with(200%)
  #canvas({
    import draw: *
    // change the mode of uncover, and its hider
    let uncover = uncover.with(mode: "array", hider: hide.with(bounds: true))

    circle((0, 0), fill: red)
    (pause,)
    circle((1, 0), fill: green)
    uncover(3, line((-2, 0), (rel: (5, 0))))
  })
]
```



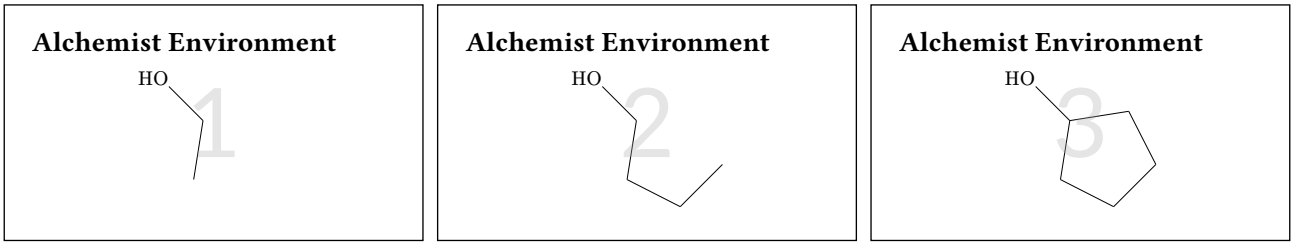

All animation functions (except `pause` and `meanwhile` which are handled automatically) that are usable in non-content environment have a `mode` argument, accepting either `"content"` or `"array"`. `"content"` is default, but you can change it so that the functions are able to join inside `array` environment, like CeTZ and Alchemist's frameworks.

However, this still has a limitation, it cannot process nested elements like in `draw.group()` of CeTZ or `cycle()` of alchemist. So, another interface, which is just a shorthand for `interface()` with the same mode of both inner data and outer environment, is introduced: `bridge()` function (because it bridges between the inner and outer workspace of the function):

```
#import "@preview/alchemist:0.2.0" as alc: skeletize
// enable Presentate's parsing
#let skeletize = interface(skeletize, inner: "array", hider: alc.hide)
#let m-cycle = bridge(alc.cycle)  // default mode is "array"

#slide[
  = Alchemist Environment
  // visuals
  #set align(center + horizon)
  #skeletize({
    import alc: *
    let uncover = uncover.with(mode: "array", hider: alc.hide)
    fragment("HO")
    single(angle: -1)
    m-cycle(5, {
      single()
      (pause,) // pauses inside `cycle`!
      single()
      single()
      (pause,)
      single()
      single()
    })
  })
]

```




## 5.2 Animate the inanimate: a custom way to hide

Presentate also has a function for handle a custom way to hide elements from other packages, which is the `animate` function in the `animation` module. This `animate` create another function that *react* with

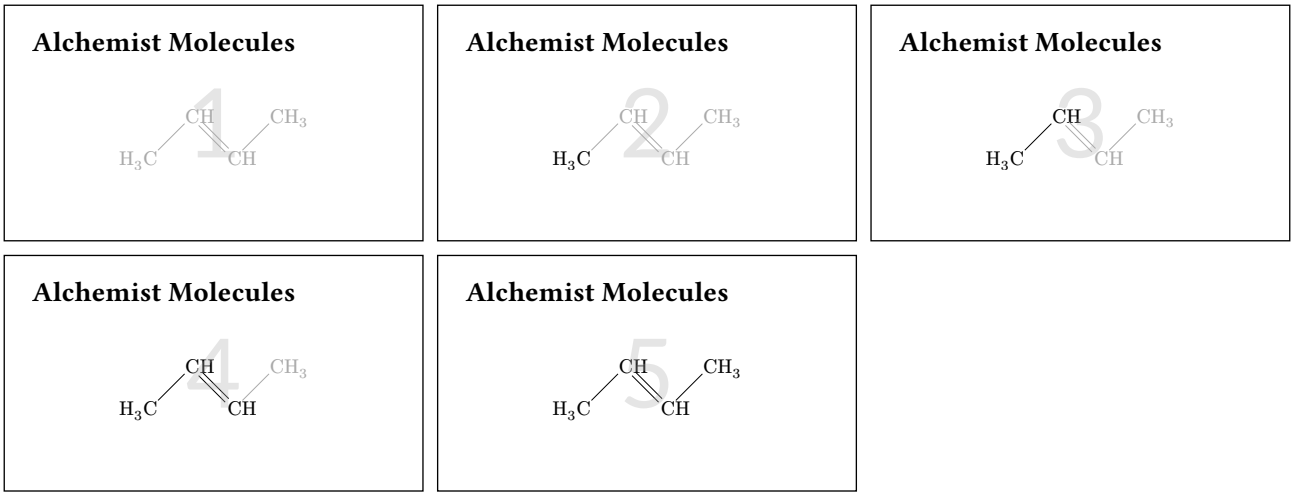
the current state `s`, so that the element will be shown one by one based on the current number of pauses, just like what normal pause do, and hidden with the specified `modifier`. To demonstrate this functionality, we will use an example to draw a chemical structure from Alchemist[8] package:

```
#import "@preview/alchemist:0.1.9" as alc

// set stroke to `gray`
#let modifier(func, ..args) = func(stroke: gray + 1pt, ..args)
#let (single, double) = animation.animate(modifier: modifier, alc.single, alc.double)
#let (fragment,) = animation.animate(
  // set atom color to gray
  modifier: (func, ..args) => func(colors: (gray,), ..args),
  alc.fragment
)

#slide[
  = Alchemist Molecules
  #set align(center + horizon)
  #render(s => (
    {
      alc.skeletize({
        fragment(s, "H_3C")
        s.push(auto)
        single(s, angle: 1)
        fragment(s, "CH")
        s.push(auto)
        double(s, angle: -1, from: 0)
        fragment(s, "CH")
        s.push(auto)
        single(s, from: 0, angle: 1)
        fragment(s, "CH_3")
      })
    },
    s,
  ))
]

```




The animate function have a `modifier` argument that receives a function that will be called when the element is hidden. In this example, the single bond is hidden by setting its stroke to `gray` and the

[8] https://typst.app/universe/package/alchemist/

atoms are hidden by setting their text color to gray. The animate can process multiple functions at the same time if those functions require the same modifier to be hidden, so `animate` will returns **an array** of *modified* functions in the same order as specified, as shown when processing `alc.single` and `alc.double` with the same modifier.

# 6 Configuration Options

You can set the following options to Presentate:

- `handout: false` to create a handout version. This option will disable all of the animation and exposed all of the hidden content.
- `drafted: false` to show the current subslide number.
- `freeze-states: true` to freeze the states and counters. Note that this functionality was inherited from Touying package, and it will freeze only the specified states and counters, which can be set in `frozen-states-and-counters` option.
- `frozen-states-and-counters` to indicate which states and counters do Presentate need to freeze. The default is

```
(
  counter(figure.where(kind: image)),
  counter(figure.where(kind: table)),
  counter(footnote),
  counter(heading),
  counter(math.equation),
)
```

All of these options can be set by

```
#set-options(..options)
```

where `options` are the options in the form `key: value`.

# 7 Exposed Utilities

## 7.1 Internal element representation

Every content and accessible elements (wrapped with `interface()` or `bridge`, for examples), are internally represented as *element*, which is a dictionary containing its construction function, arguments, and reactive properties such as ability to be hidden, inner hider function, or ability to change the parsing mode. These elements are wrapped with `element.mode-wrapper(mode, element)` which returns a metadata or an array in `"content"` or `"array"` mode, respectively, so that the elements can be joined natively to their surrounding elements.

## 7.2 Internal states

Internally, Presentate store all of the slides information in a state in the `store` module. You can access this state by

```
store.getter(states ⇒ { .. })
```

or if you are outside of `slide()`,

```
context store.states.get()
```

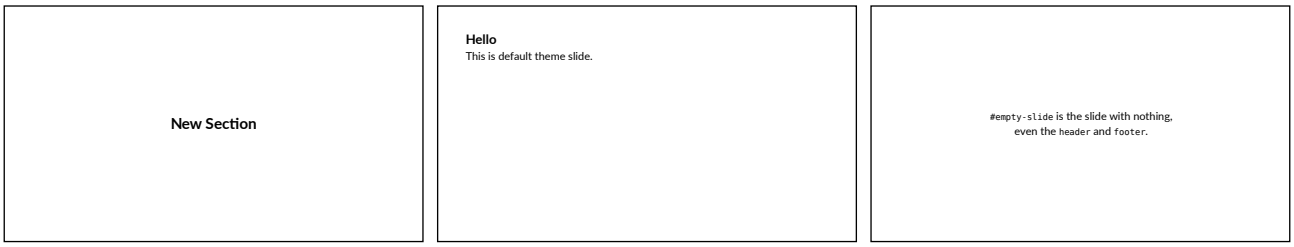
The state's information will be an array of indices with the global info in the first member of this array. It is the same `s` as you can access from `render` or `motion` function.

# 8 Themes

There are 2 types of themes that Presentate provides:

1. **normal type**, which is just a decoration of the slide,
   - **default** theme, just a normal, plain presentation,
   - **simple** theme, a decorative theme with auto heading continuation,
   - **classic** theme, a plain theme that slides can inherit the same title from the current level 2 heading and auto fit-to-height image insertion.
2. **structured themes**, which contains a functionality from Navigator[9] package. These amazing themes are created by David Hajage, which has a complete guide of how to use the themes in theme guide.

### Default Theme




```typst
#import themes.default: *

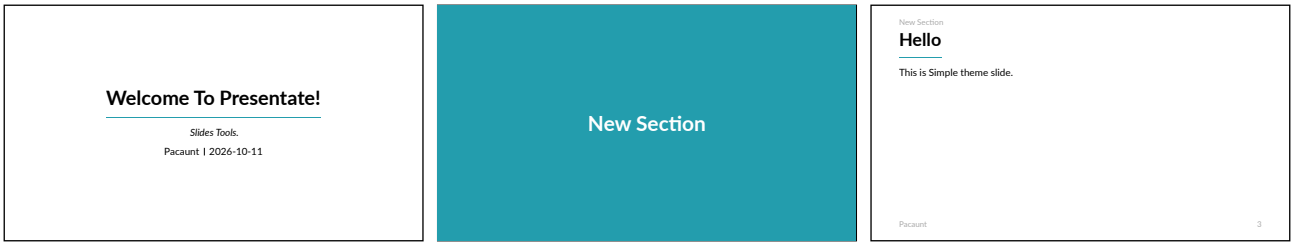
#show: template.with(
  aspect-ratio: "16-9"
)

= New Section

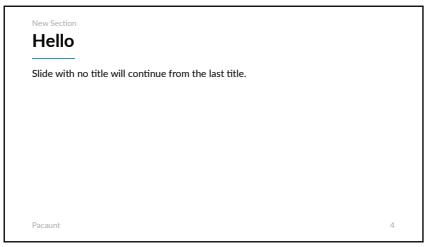
#slide[
  == Hello
  This is default theme slide.
]

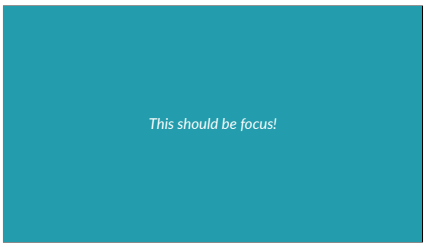
#empty-slide[
  #set align(center + horizon)
  `#empty-slide` is the slide with nothing, \
  even the `header` and `footer`.
]
```

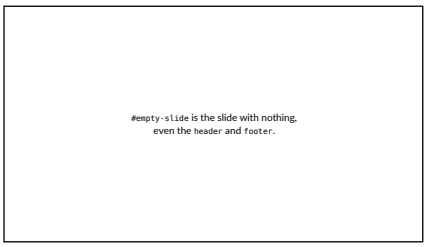
### Simple Theme




[9] https://typst.app/universe/package/navigator








```typ
#import themes.simple: *

#show: template.with(
  author: [Pacaunt], // change to yours!
  title: [Welcome To Presentate!],
  subtitle: [Slides Tools.],
)

= New Section

#slide[Hello][
  This is Simple theme slide.
]

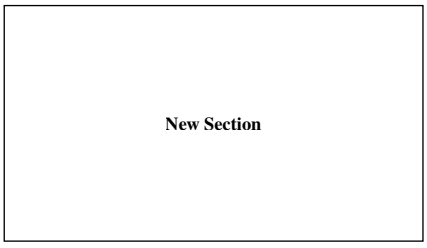
#slide[
  Slide with no title will continue from the last title.
]

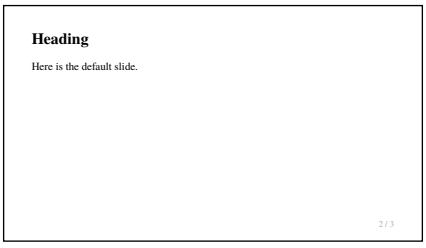
#focus-slide[
  This should be focus!
]

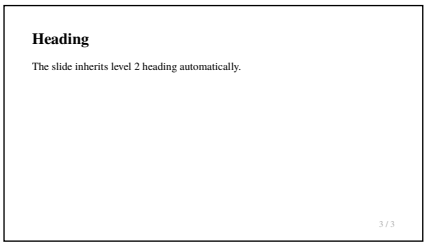
#empty-slide[
  #set align(center + horizon)
  `#empty-slide` is the slide with nothing, \
  even the `header` and `footer`.
]


```

## Classic Theme








```typ
#import themes.classic: *

#show: template.with(
  aspect-ratio: "16-9",
)

= New Section

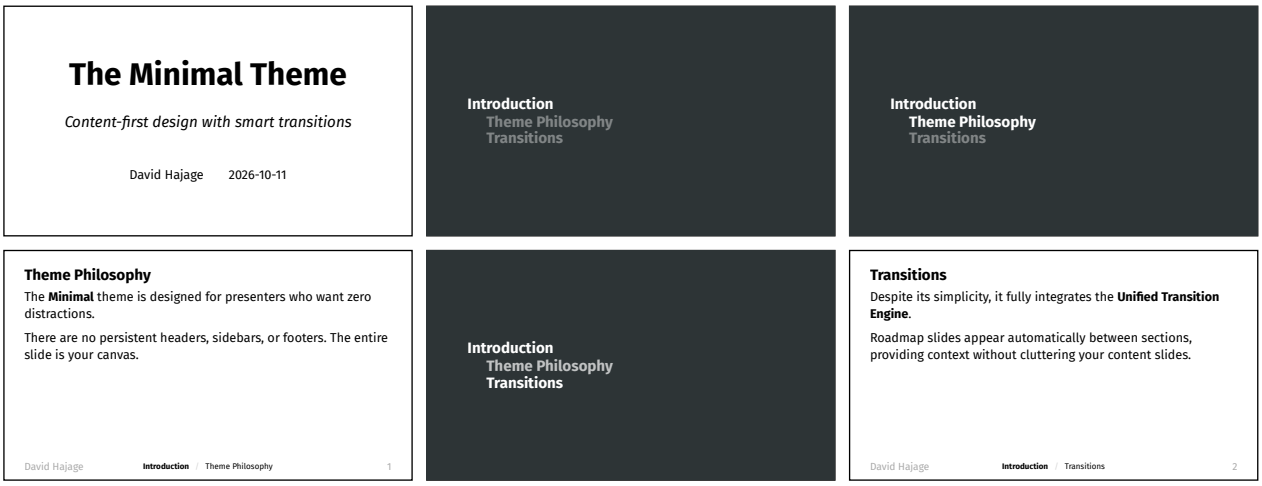
== Heading
```

```
10
11 #slide[
12   Here is the default slide.
13 ]
14
15 #slide[
16   The slide inherits level 2 heading automatically.
17 ]
```

### Structured Themes

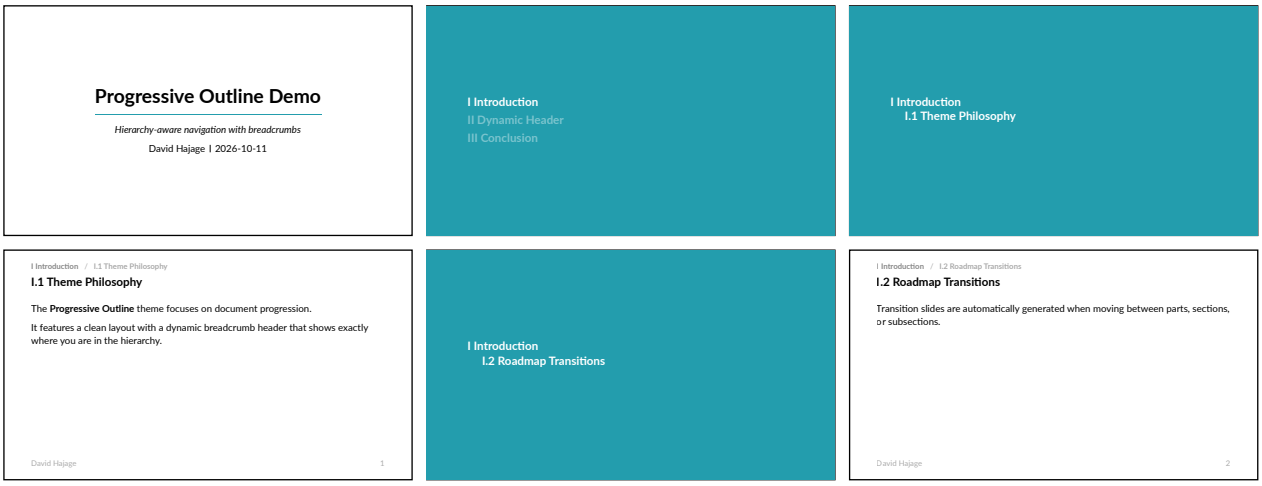
You can visit the examples of the structured themes here:

- **minimal**



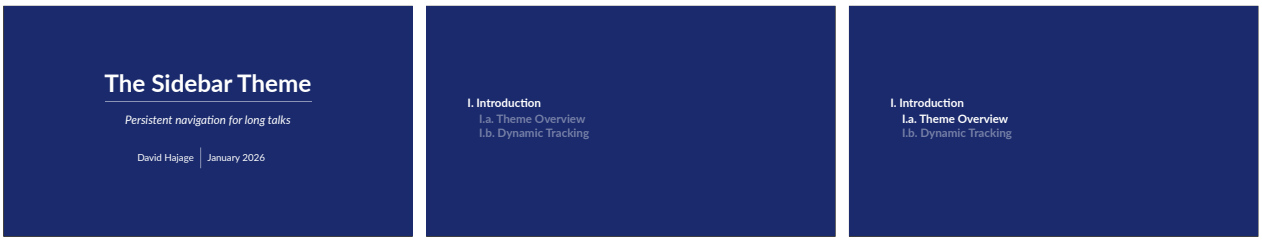

Source

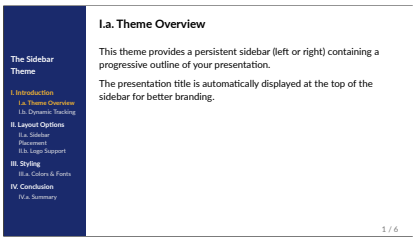
- **progressive-outline**

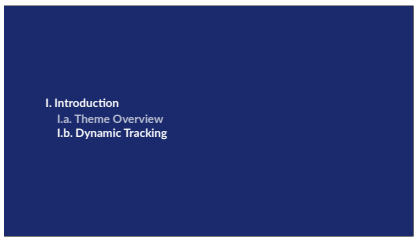


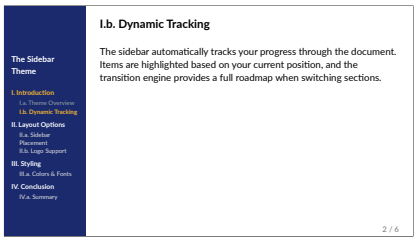

Source

- **sidebar**





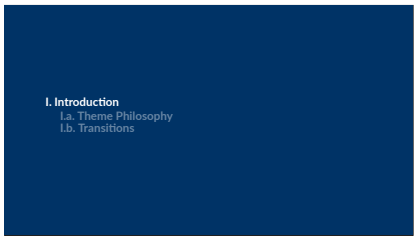


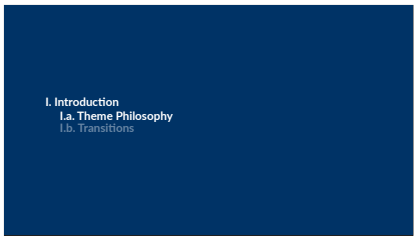


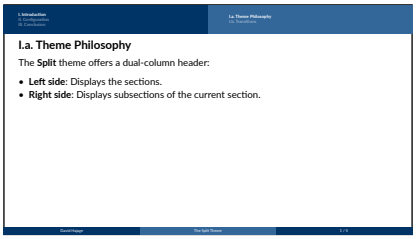

Source

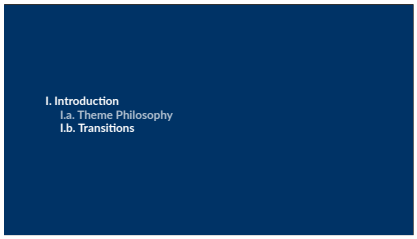
- **split**

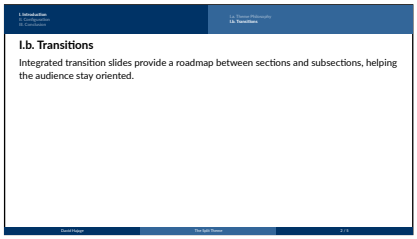






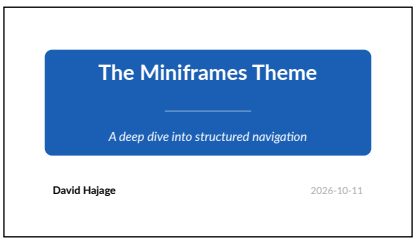


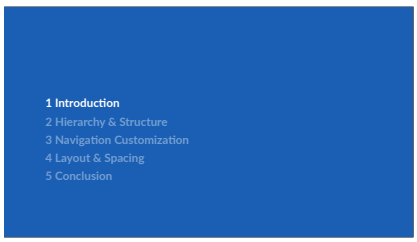


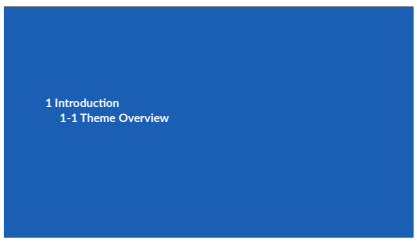


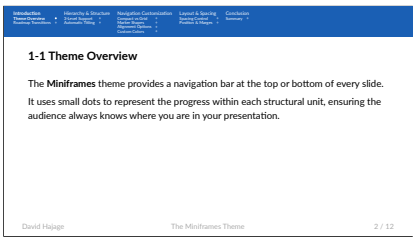

Source

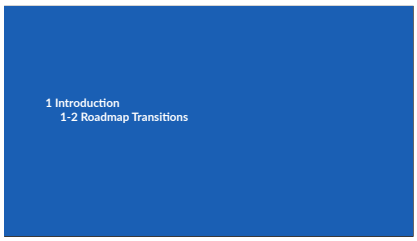
- **miniframes**

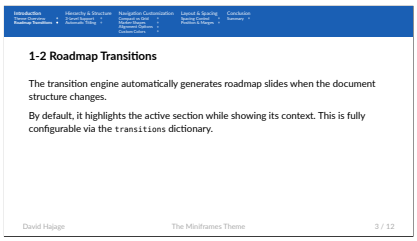













Source

# 9 Function Reference

## 9.1 Animation Functions

- alert()
- fragments()
- motion()
- only()
- render()
- reveal-item()

- step-item()
- tag()
- transform()
- uncover()

## alert

Alert a text to make it pop.

**Parameters**

```
alert(
  ..n: index,
  from: index,
  to: index,
  body: content,
  func: function,
  update: bool,
  mode: "content" "array"
) -> content
```

**..n** `index`

indices to alert.

**from** `index`

start index to alert

Default: `()`

**to** `index`

stop index to alert

Default: `()`

**body** `content`

the content

**func** `function`

alerted function, default is Typst's emphasis function.

Default: `emph`

**update** `bool`

whether to update the current number of pauses

Default: `false`

**mode** `"content"` or `"array"`

mode of using this function

Default: `"content"`

## `fragments`

Show the content one by one.

**Parameters**

```
fragments(
  start: index,
  ..bodies: content,
  hider: function,
  update: bool,
  update-increment: bool,
  item-wrapper,
  mode: "content" "array"
) -> content
```

**start** `index`

animation start index

Default: `auto`

**..bodies** `content`

contents to be revealed.

**hider** `function`

hiding function

Default: `hide`

**update** `bool`

whether to update the current number of pauses.

Default: `true`

**update-increment** `bool`

whether to update the current number of pauses for each content inside.

Default: `true`

**item-wrapper**

A function wrapper that can wrap the content.

Default: `it ⇒ it`

**mode** `"content"` or `"array"`

mode of using this function

Default: `"content"`

## motion

Motion workspace. This function allows user to control the presence and modify the content of each tags directly for each subslide.

```
#motion(s ⇒ [
  // your content with tags
], controls: (
  .. // an array of rules indicating what to be shown
))
```

### Parameters

```
motion(
  func: function,
  controls: array,
  hider: function,
  start: index,
  update: bool,
  is-shown: bool,
  mode
) -> content
```

**func** `function`

A function that receives Presentate's state `s` and returns a content.

**controls** `array`

This is an array of motion control. `(A, B, C)` means show `A` then `B` then `C`. `(A, (B, C), C)` means show `A`, then `B` + `C`, and then `C`.

Default: `()`

**hider** `function`

default hider for contents in the tags

Default: `hide`

**start** `index`

start index of the animation

Default: `none`

**update** `bool`

whether to update the pauses after the animation

Default: `false`

**is-shown** `bool`

whether to show the content in the tags by default

Default: `false`

## only

Show content on specific subslide without preserving space.

**Parameters**

```
only(
  ..n: index,
  body: content,
  from: index,
  to: index,
  hider: function,
  update: bool,
  mode: "content" "array"
) -> content
```

**..n** `index`

indices to show the content.

**body** `content`

the content.

**from** `index`

index to start showing the content.

Default: `()`

**to** `index`

index to stop showing the content.

Default: `()`

**hider** `function`

hiding function

Default: `it => none`

**update** `bool`

whether to update the current number of pauses.

Default: `false`

**mode** `"content"` or `"array"`

mode of using this function

Default: `"content"`

## render

Workspace for creating animation by accessing Presentate's internal states. Use with the animation module.

```
#render(s => ({
  import animation: *
  // your content
}, s))
```

### Parameters

```
render(
  start: index,
  func: function
)
```

**start** `index`

When to start showing the rendered content.

Default: `auto`

**func** `function`

A function that returns an array of length two consisting of the rendered content and the updated state `s`.

## `reveal-item`

Reveal the item group by group.

**Parameters**

```
reveal-item(
  start: index,
  numbering: function str,
  marker,
  hider: function,
  accumulated: bool,
  ..args: enum list
) -> content
```

**start** `index`

start index of the animation

Default: `auto`

**numbering** `function` or `str`

numbering for enums. `auto` means inherting from the current style of `enum`.

Default: `auto`

**marker**

marker for lists. `auto` means inheriting from the current style of `list`.

Default: `auto`

**hider** `function`

hider for the list/enums

Default: `hide`

**accumulated** `bool`

whether to show the shown list/enum items. If set to `false`, each list/enum item will be shown only once per animation.

Default: `true`

**..args** `enum` or `list`

the enum/list

## step-item

Incrementally show items in enums/lists. This animation always update the current number of pauses.

**Parameters**

```
step-item(
  body: enum list,
  start: index,
  numbering: function str,
  marker,
  hider: function,
  ..args: any,
  lead-parbreak
) -> content
```

**body** `enum` or `list`

The list/enum. Must not contains any set/show rules.

**start** `index`

start index of the animation

Default: `auto`

**numbering** `function` or `str`

numbering for enums. `auto` means inherting from the current style of `enum`.

Default: `auto`

**marker**

marker for lists. `auto` means inheriting from the current style of `list`.

Default: `auto`

**hider** `function`

hider for the list/enums

Default: `hide`

**..args** `any`

other styling arguments will be passed to enum/list set rules.

## `tag`

Use with the `motion` function. Tagging the content into a group with a name for animating with `motion`'s `controls` rules.

**Parameters**

```
tag(
  s: presentate-state,
  name: str,
  body: any,
  hider: function auto,
  func: function
) -> any
```

**s** `presentate-state`

Presentate's state. Must be from `motion` workspace.

**name** `str`

name of the group

**body** `any`

the content

**hider** `function` or `auto`

the hider used to hide the content. If this is set to `auto`, the hider will inherits from `motion` workspace.

Default: `auto`

**func** `function`

default content wrapper

Default: `it ⇒ it`

# transform

Transform a content by passing it to some functions.

**Parameters**

```
transform(
  start: index,
  body: content,
  ..funcs: function,
  repeat-last: bool,
  hider: function,
  update: bool,
  mode: "content" "array"
) -> content
```

**start** `index`

start showing index

Default: `auto`

**body** `content`

the content

**..funcs** `function`

the functions,

**repeat-last** `bool`

whether to keep the last animated result when all of the animation has been shown.

Default: `true`

**hider** `function`

hiding function

Default: `hide`

**update** `bool`

whether to update the current number of pauses.

Default: `true`

**mode** `"content"` or `"array"`

mode of using this function

Default: `"content"`

# uncover

Reveal content on specific subslide, with space preserved.

**Parameters**

```
uncover(
  ..n: index,
  body: content,
  from: index,
  to: index,
  hider: function,
  update: bool,
  mode: "content" "array"
) -> content
```

**..n** `index`

indices to show the content.

**body** `content`

the content.

**from** `index`

index to start showing the content.

Default: `()`

**to** `index`

index to stop showing the content.

Default: `()`

**hider** `function`

hiding function

Default: `auto`

**update** `bool`

whether to update the current number of pauses.

Default: `false`

**mode** `"content"` or `"array"`

mode of using this function

Default: `"content"`

## 9.2 Package Integration Interface


- bridge()
- interface()
- labeler()


### `bridge`

Another custom interface, used within the `interface`, i.e. when the inner and outer mode is the same. You can use this to wraps around `context` in content mode.

**Parameters**

```
bridge(
  func: function,
  mode: "array" "content",
  ..props
) -> function
```

**func** `function`

The sub-environment function, such as `group()` in CeTZ or `branch()` in alchemist package.

**mode** `"array"` or `"content"`

Mode of parsing, which determines the 'kind' of content inside.

Default: `"array"`

### `interface`

A custom interface that makes Presentate reach the content inside.

**Parameters**

```
interface(
  func: function,
  inner: "array" "content",
  outer: "array" "content",
  hider: function auto
) -> function
```

**func** `function`

The environment function. Such as `cetz.canvas`.

**inner** `"array"` or `"content"`

Inner mode of parsing

Default: `"array"`

**outer** `"array"` or `"content"`

Outer mode of parsing, i.e. the environment surrounding this element.

Default: `"content"`

**hider** `function` or `auto`

Default hider used by `pause`

Default: `auto`

## `labeler`

Label contents such that the label will only appear once. Example usage:

```
#labeler(
  figure(image("example.png"), caption: [A caption.]),
  <name>
)
From @name, it is an example.
```

**Parameters**

```
labeler(
  body: content,
  name: label
) -> content
```

**body** `content`

The content to get labeled.

**name** `label`

The label

## 9.3 Presentate Slide Function

- slide()

## `slide`

Presentate slide function

**Parameters**

```
slide(
  body: content,
  steps: auto int,
  body-fn: function,
  preamble: function,
  logical-slide: bool,
  animated: auto bool
) -> content
```

**body** `content`

the content on the slide

**steps** `auto` or `int`

Total number of subslides needed. `auto` means detecting automatically.

Default: `auto`

**body-fn** `function`

function that wraps only the content.

Default: `it ⇒ it`

**preamble** `function`

function that wraps the whole slide.

Default: `it ⇒ it`

**logical-slide** `bool`

whether this should be counted as a logical slide. This effects page numbering as non-logical slide will have a skipped page number.

Default: `true`

**animated** `auto` or `bool`

whether to enable the animation on this slide. Crucial for themes.

Default: `true`

## 9.4 Special Function in `animation` module

- animate()

**`animate`**

Modify the function so that it can react to the state variable `s` Default behaviour is like `pause`

### Parameters

```
animate(
  ..funcs: function,
  wrapper,
  hider: function,
  modifier: function none
)
```

**..funcs** `function`

The orignial functions

**hider** `function`

hiding function

Default: `it ⇒ none`

**modifier** `function` or `none`

modifier for hiding the content. if it is not `none` then it must be `(func, ..args) ⇒ ..`

Default: `none`